

NoSQL: Familia de columnas

Dr. Luis Gustavo Esquivel Quirós
Escuela de Ciencias de la Computación e Informática
Universidad de Costa Rica

Adaptado de versión original de
Aidan Hogan

Familia de columnas

- Una familia de columnas es similar a una columna en un DBMS relacional .
- Ambas se definen típicamente en tablas antes de almacenar los datos y son de naturaleza estática, con la diferencia que las columnas en una base de datos relacional se definen con el tipo de datos que pueden almacenar, mientras que las familias de columnas pueden almacenar cualquier tipo de datos, siempre que puedan conservarse como una matriz de bytes.
- Además, cada fila en una familia de columnas almacena valores de datos solo en aquellas columnas para las que tiene valores válidos.

Familia de columnas

- Big table y Dynamo se consideran precursores de las bases de datos de la familia de columnas.
- Normalmente las columnas que a menudo no se consultan juntas se distribuyen en diferentes nodos.
- Un almacenamiento orientado a columnas se utiliza en aplicaciones analíticas para mejorar el rendimiento.

Key–Value = a Distributed Map

Countries	
Primary Key	Value
Afghanistan	capital:Kabul,continent:Asia,pop:31108077#2011
Albania	capital:Tirana,continent:Europe,pop:3011405#2013
...	...

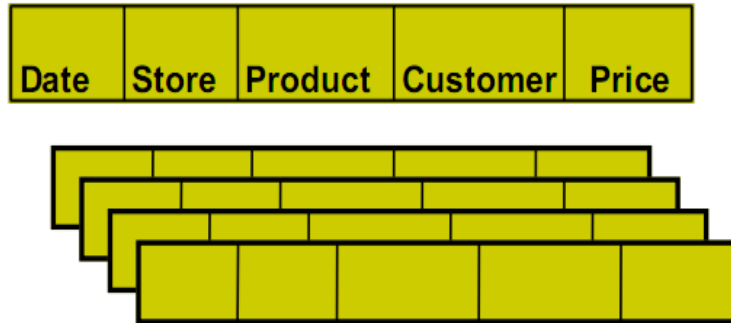
Tabular = Multi-dimensional Maps

Countries				
Primary Key	capital	continent	pop-value	pop-year
Afghanistan	Kabul	Asia	31108077	2011
Albania	Tirana	Europe	3011405	2013
...	...	...	...	...

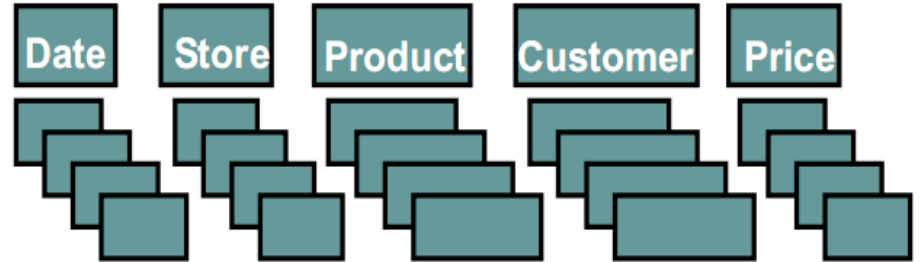
Familia de columnas

- El almacenamiento de los datos se realiza columna por columna en el disco.

row-store



column-store



Familia de columnas

- Las familias de columnas tienen la ventaja de solo leer los datos relevantes.
- Pero tienen el defecto que requieren múltiples accesos para escribir una tupla.
- Poseen mejor compresión de datos.
- Pueden trabajar sobre datos comprimidos.

Modelo de datos

- Es muy similar al modelo relacional con: Tablas, filas y columnas.
- Claves primarias y claves foráneas
- Proyecciones son para una sola tabla.

Bigtable: The Original Whitepaper

Autores de
MapReduce

Bigtable: A Distributed Storage System for Structured Data

Fay Chang, Jeffrey Dean, Sanjay Ghemawat, Wilson C. Hsieh, Deborah A. Wallach,
Mike Burrows, Tushar Chandra, Andrew Fikes, Robert E. Gruber
{fay,jeff,sanjay,wilsonh,kerr,m3b,tushar,fikes,gruber}@google.com

Google, Inc.

Abstract

Bigtable is a distributed storage system for managing structured data that is designed to scale to a very large size: petabytes of data across thousands of commodity servers. Many projects at Google store data in Bigtable, including web indexing, Google Earth, and Google Finance. These applications place very different demands on Bigtable, both in terms of data size (from URLs to web pages to satellite imagery) and latency requirements (from backend bulk processing to real-time data service). Despite these varied demands, Bigtable has successfully provided a flexible, high-performance solution for all of these Google products. In this paper, we describe the simple data model provided by Bigtable, which gives clients dynamic control over data layout and format, and we describe the design and implementation of Bigtable.

achieved scalability and high performance, but Bigtable provides a different interface than such systems. Bigtable does not support a full relational data model; instead, it provides clients with a simple data model that supports dynamic control over data layout and format, and allows clients to reason about the locality properties of the data represented in the underlying storage. Data is indexed using row and column names that can be arbitrary strings. Bigtable always treats data as uninterpreted strings, although clients often marshal various forms of structured and semi-structured data into these strings. Clients can control the locality of their data through careful choices in their schemas. Finally, Bigtable schema parameters let clients dynamically control whether to serve data out of memory or from disk.

Section 2 describes the data model in more detail, and Section 3 provides an overview of the client API. Sec-

Bigtable es usado por ...



Bigtable: Modelo de datos

“Google Big Table es un **mapa** ordenado **multidimensional persistente**, **distribuido** y **disperso**” (Chang et al., 2006)

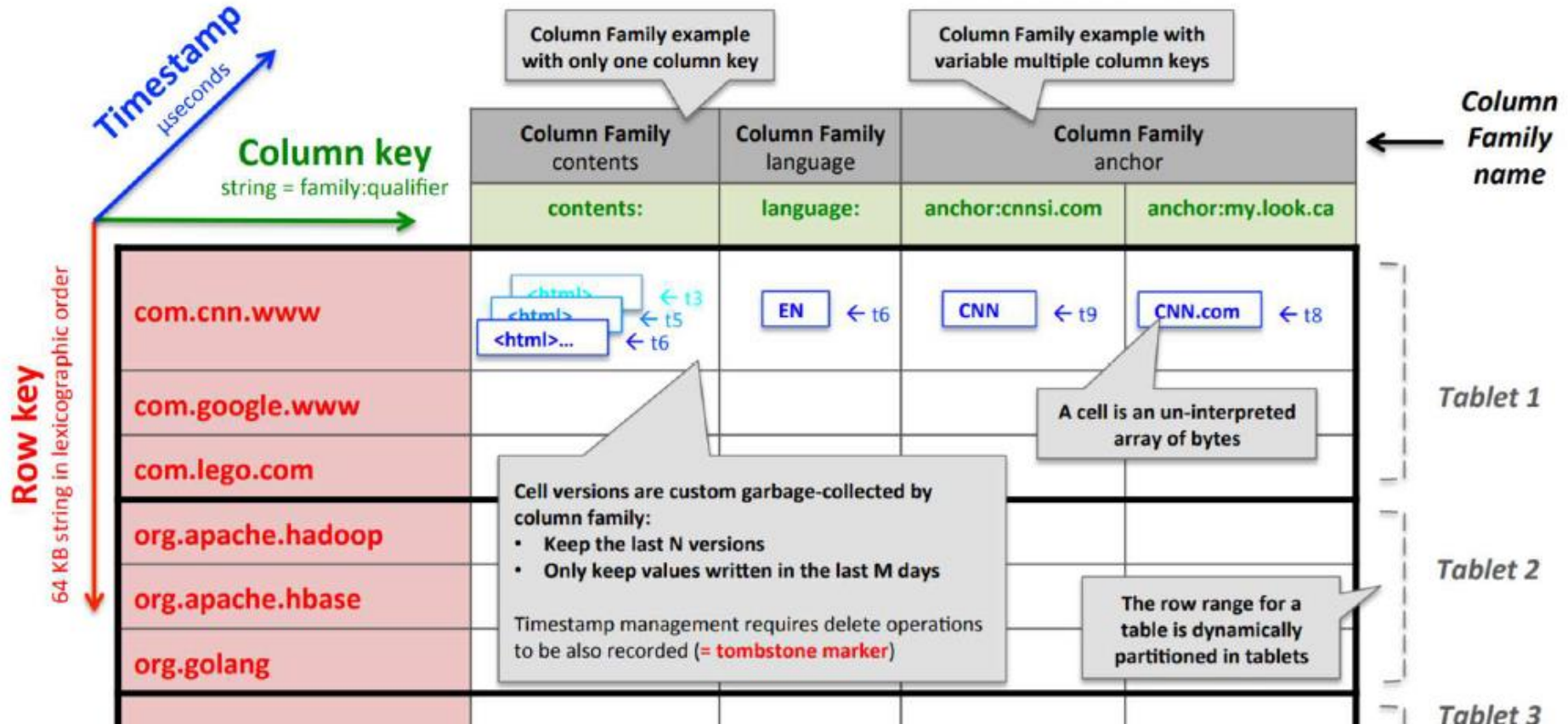
- **disperso**: la tabla es dispersa, lo que significa que diferentes filas de una tabla pueden usar diferentes columnas, con muchas de las columnas vacías para una fila en particular
- **distribuido**: los datos se distribuyen entre muchas máquinas independientes, agrupando filas adyacentes y ubicándolas en un servidor separado
- **persistente**: los datos se almacenan de forma persistente en el disco

Bigtable: Modelo de datos

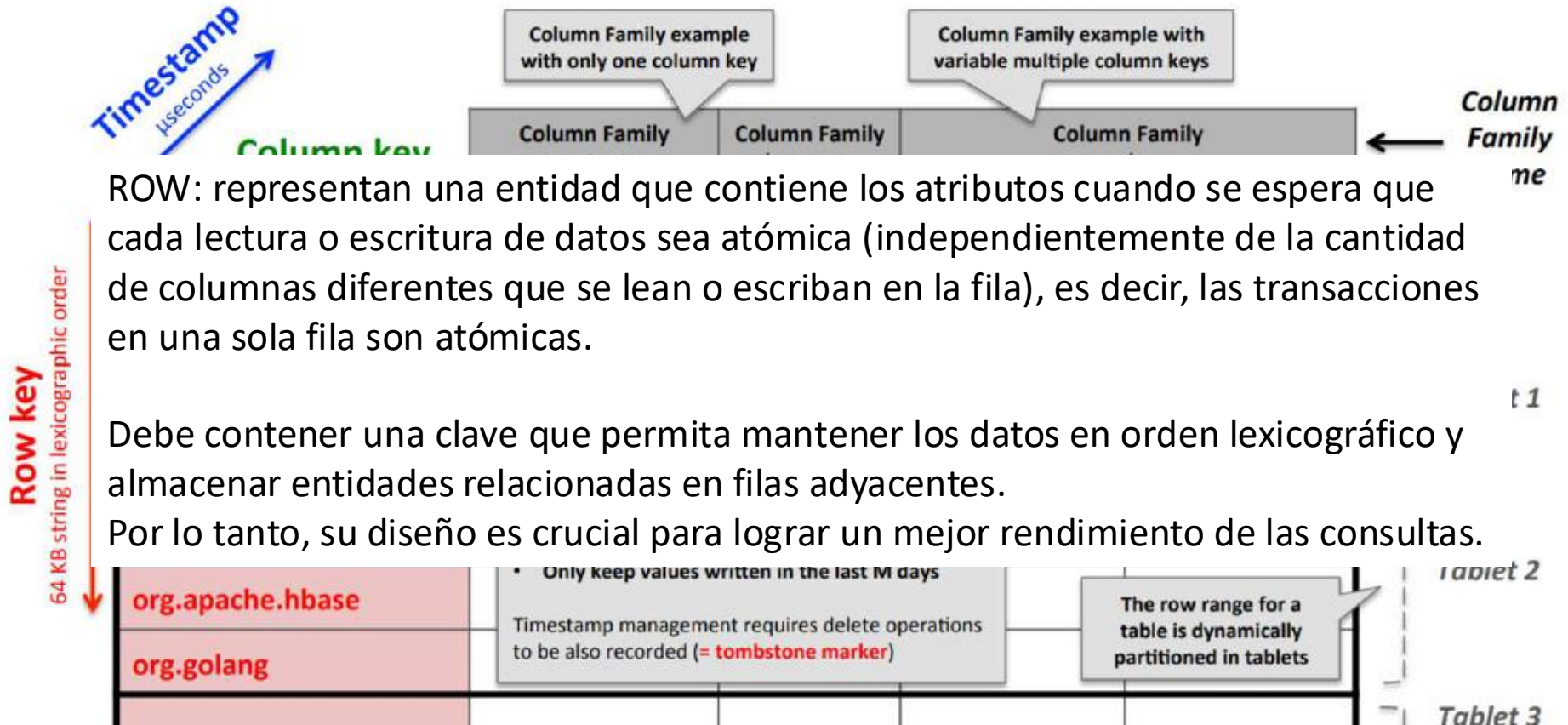
“Google Big Table es un **mapa ordenado multidimensional persistente, distribuido y disperso**” (Chang et al., 2006)

- **ordenado**: la mayoría de las matrices asociativas no están ordenadas. Ordena sus datos por claves (suponiendo una estructura de claves que por medio de esa ordenación reúna los datos). Esto ayuda a mantener juntos los datos relacionados, generalmente en la misma máquina. Por ejemplo, si se utilizan nombres de dominio como claves, se recomienda almacenarlos en orden inverso para garantizar que los dominios relacionados estén juntos, por ejemplo, cr.ac.ucr.ecci.plataforma, cr.ac.ucr.ecci.averias, cr.ac.ucr.ecci.
- **multidimensional**: cada fila contiene una o más familias de columnas nombradas
- **mapa**: es una matriz asociativa, es decir, una estructura de datos que permite buscar rápidamente un valor para una clave correspondiente. Bigtable es una colección de pares (clave, valor) donde la clave identifica una fila y el valor es el conjunto de columnas agrupadas en la llamada familia de columnas.

Bigtable: Modelo de datos



Bigtable: Modelo de datos

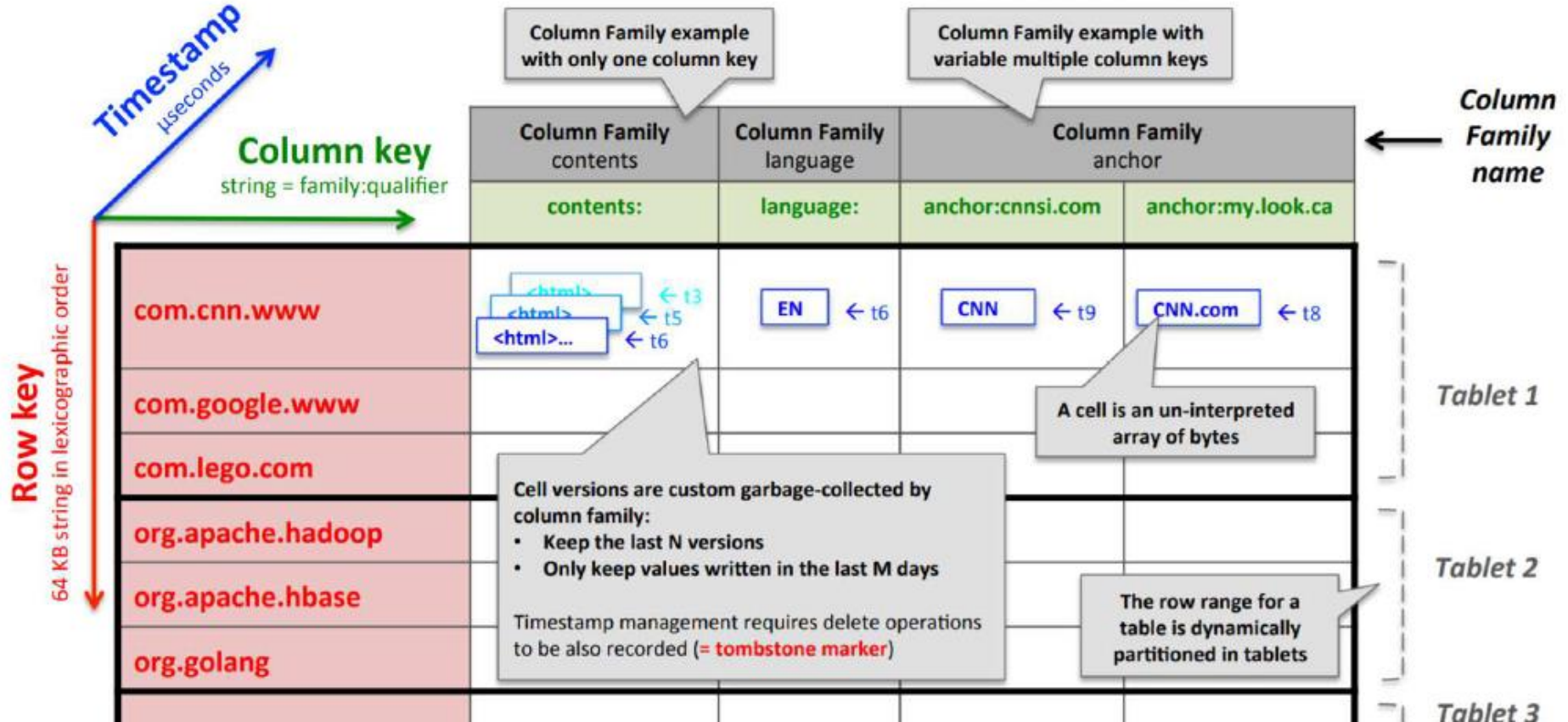


ROW: representan una entidad que contiene los atributos cuando se espera que cada lectura o escritura de datos sea atómica (independientemente de la cantidad de columnas diferentes que se lean o escriban en la fila), es decir, las transacciones en una sola fila son atómicas.

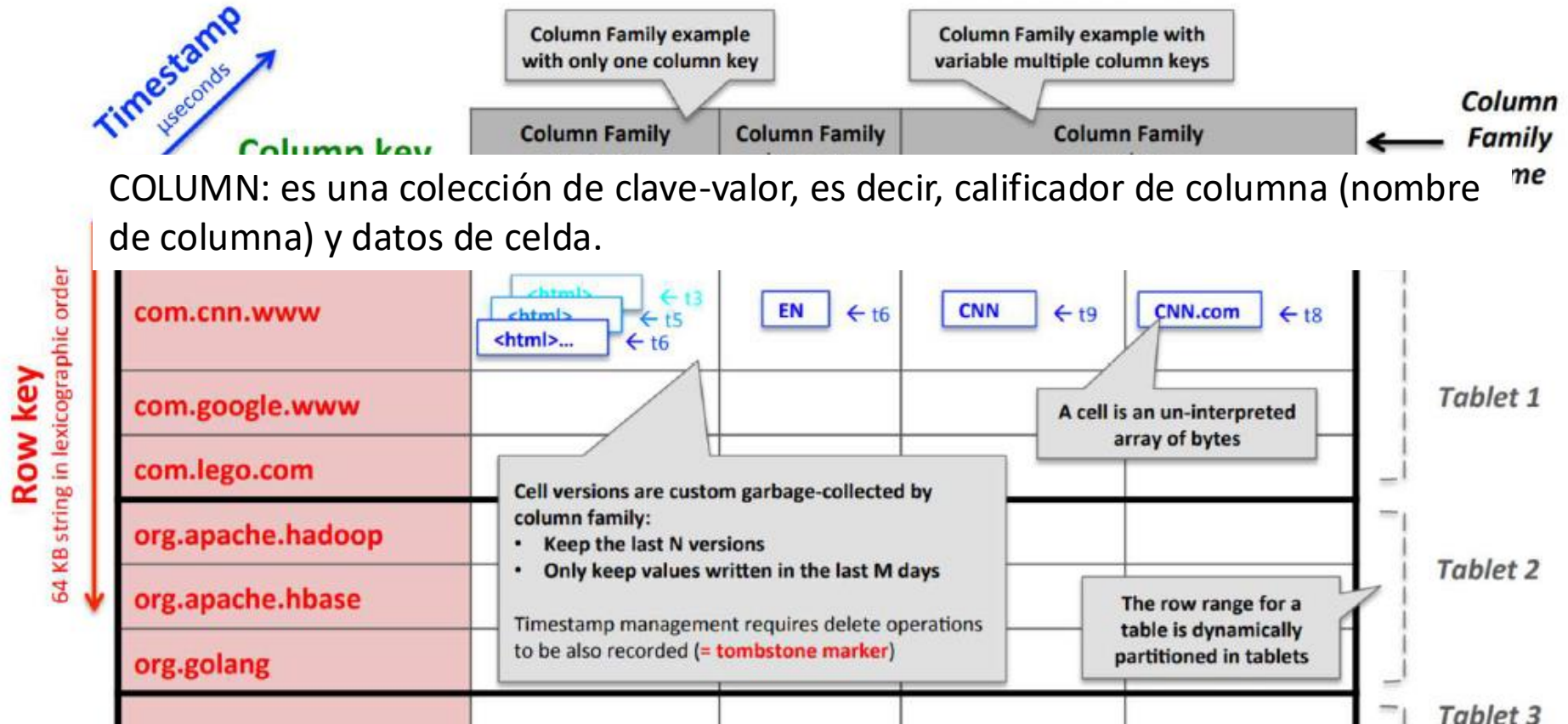
Debe contener una clave que permita mantener los datos en orden lexicográfico y almacenar entidades relacionadas en filas adyacentes.

Por lo tanto, su diseño es crucial para lograr un mejor rendimiento de las consultas.

Bigtable: Modelo de datos

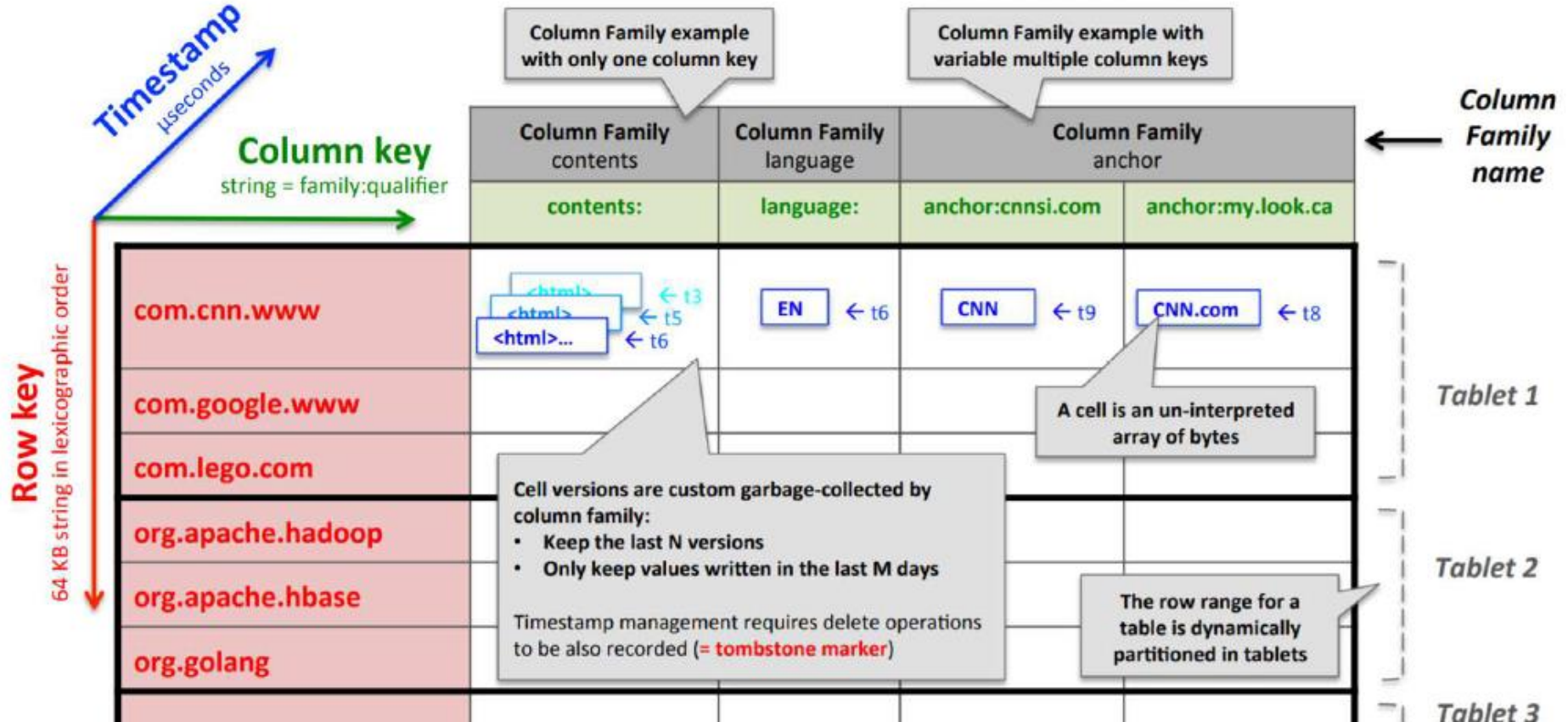


Bigtable: Modelo de datos

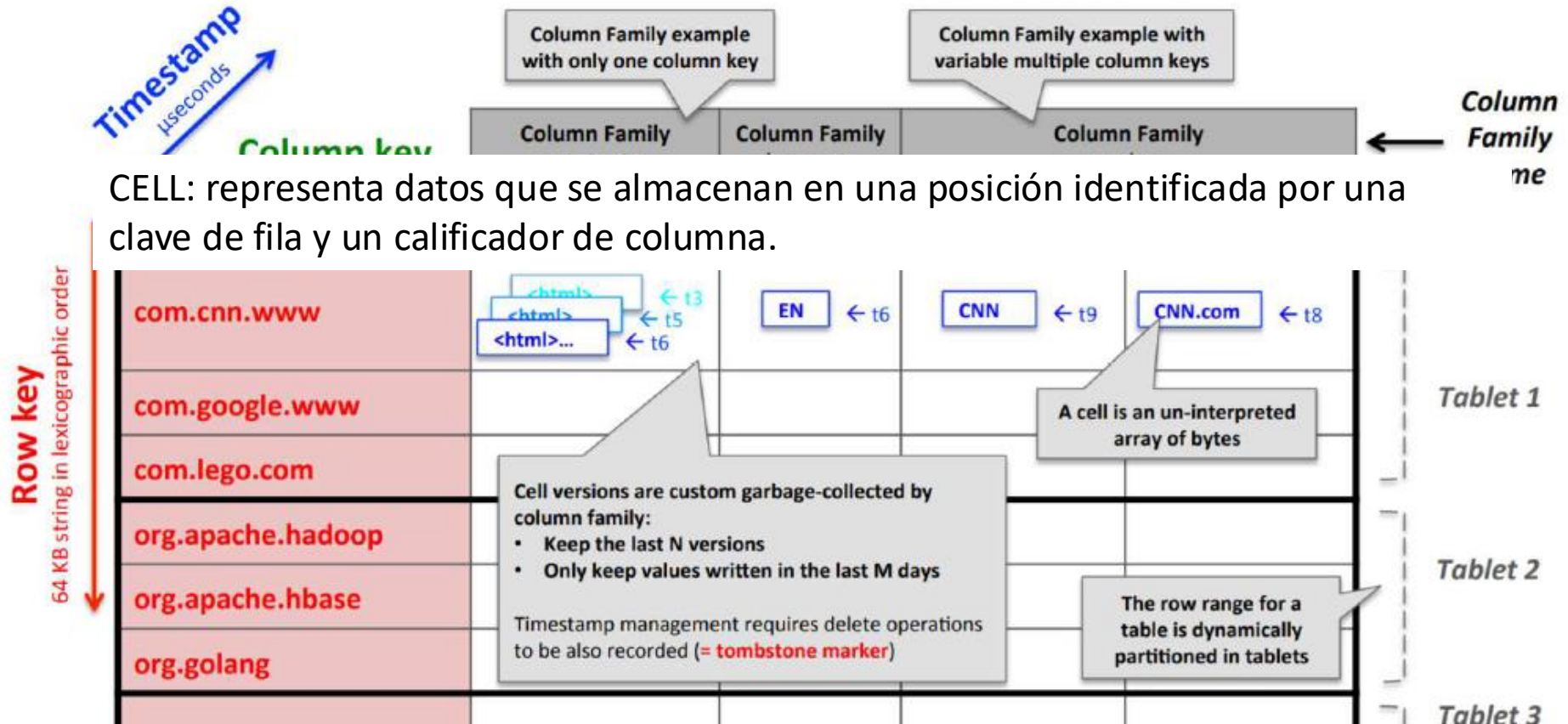


COLUMN: es una colección de clave-valor, es decir, calificador de columna (nombre de columna) y datos de celda.

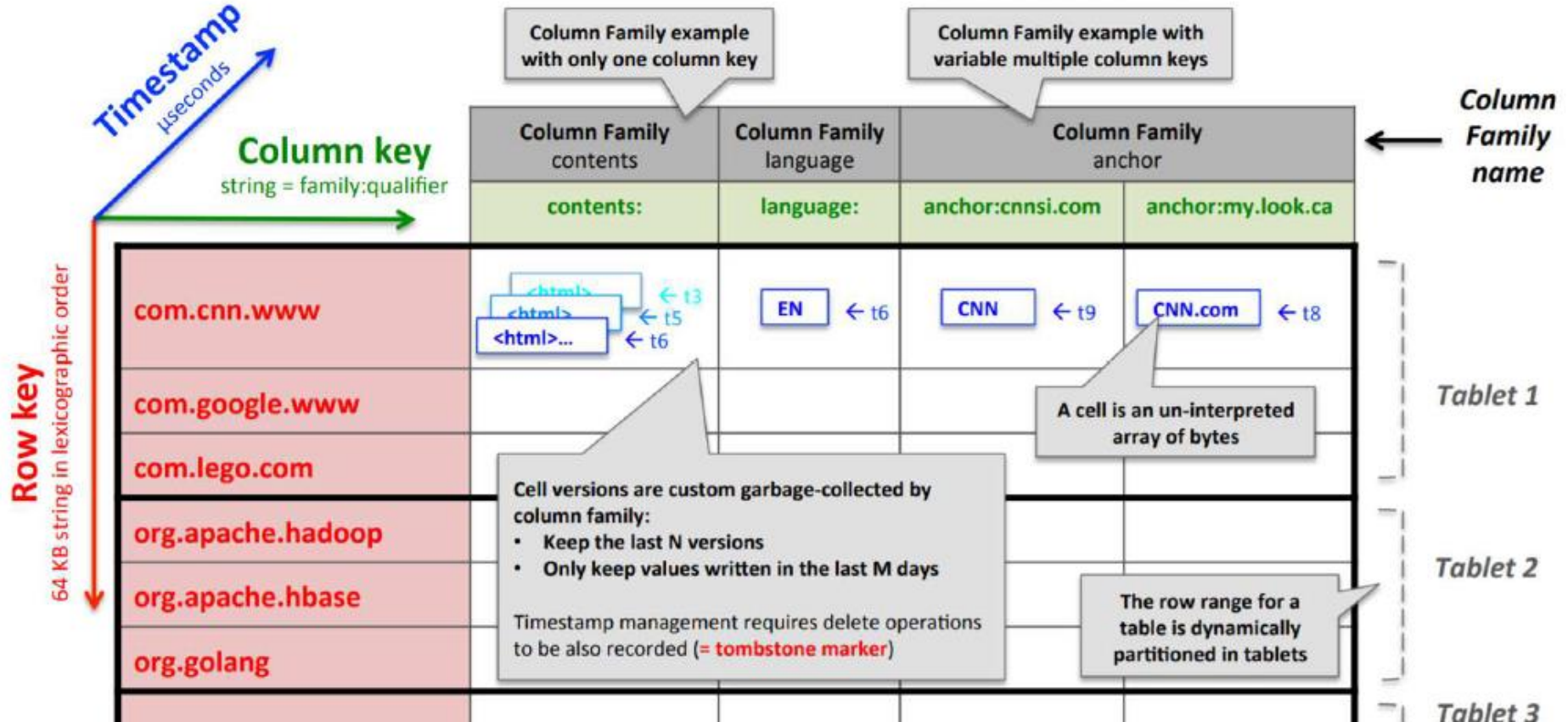
Bigtable: Modelo de datos



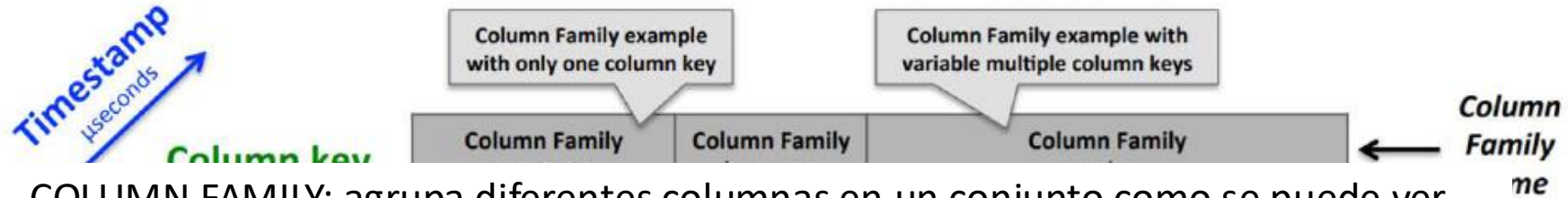
Bigtable: Modelo de datos



Bigtable: Modelo de datos



Bigtable: Modelo de datos



COLUMN FAMILY: agrupa diferentes columnas en un conjunto como se puede ver en la figura para las familias de columnas; esto se hace para garantizar un mejor rendimiento al recuperar solo los datos de interés como familias de columnas, en lugar de todos los datos que contiene una fila. Dentro de una familia de columnas, uno puede tener una o más columnas con nombre (calificadores). Todos los datos almacenados en una familia de columnas suelen ser del mismo tipo. Se debe crear una familia de columnas antes de poder incluir los datos; una vez que se ha creado una familia, se puede usar cualquier clave de columna (calificador) dentro de la familia. El número de familias de columnas distintas en una tabla puede ser de aproximadamente 100 en la versión actual de la tabla Cloud Big (Google, 2018b), y esas familias rara vez deberían cambiar durante la operación. Por el contrario, una tabla puede tener un número ilimitado de columnas.

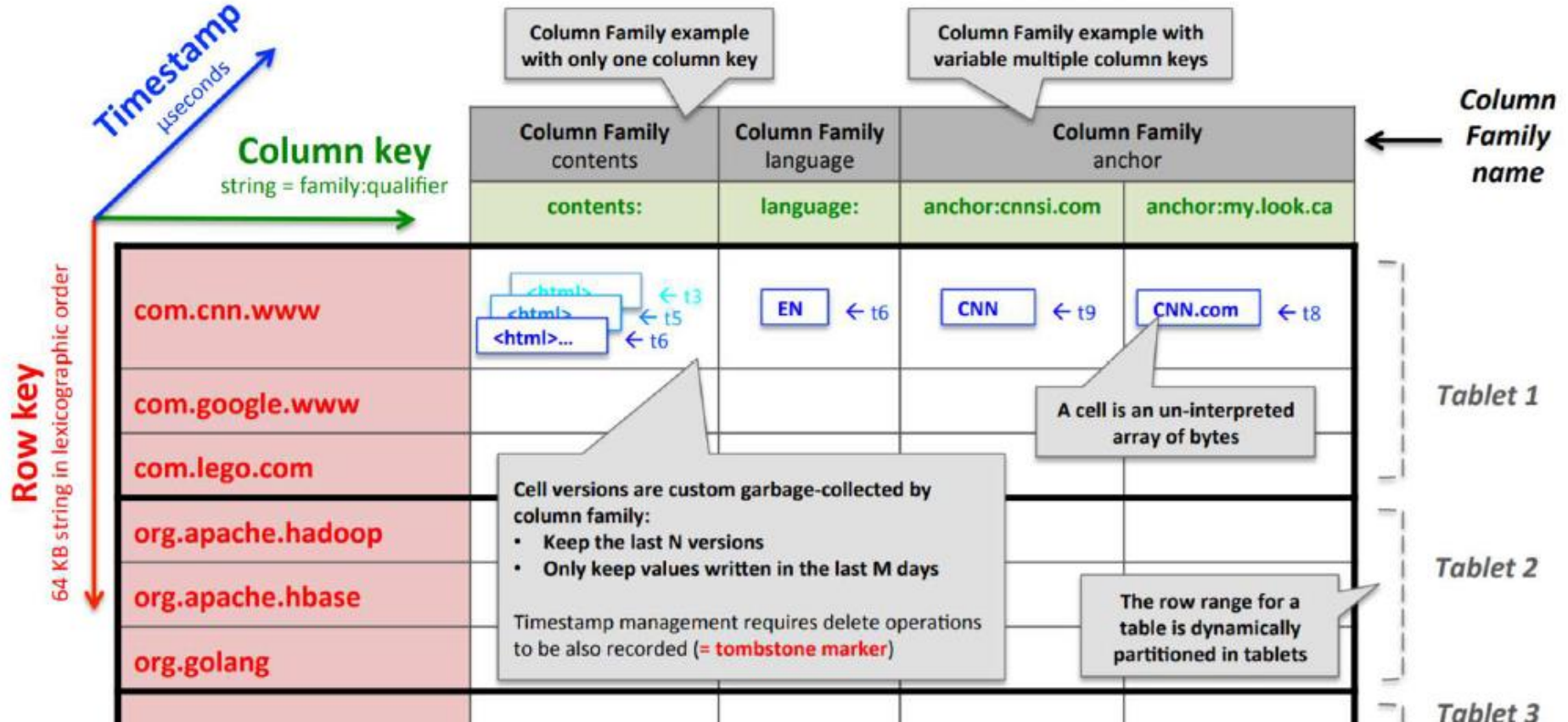
Row key
64 KB string in lexicographic order

t 1

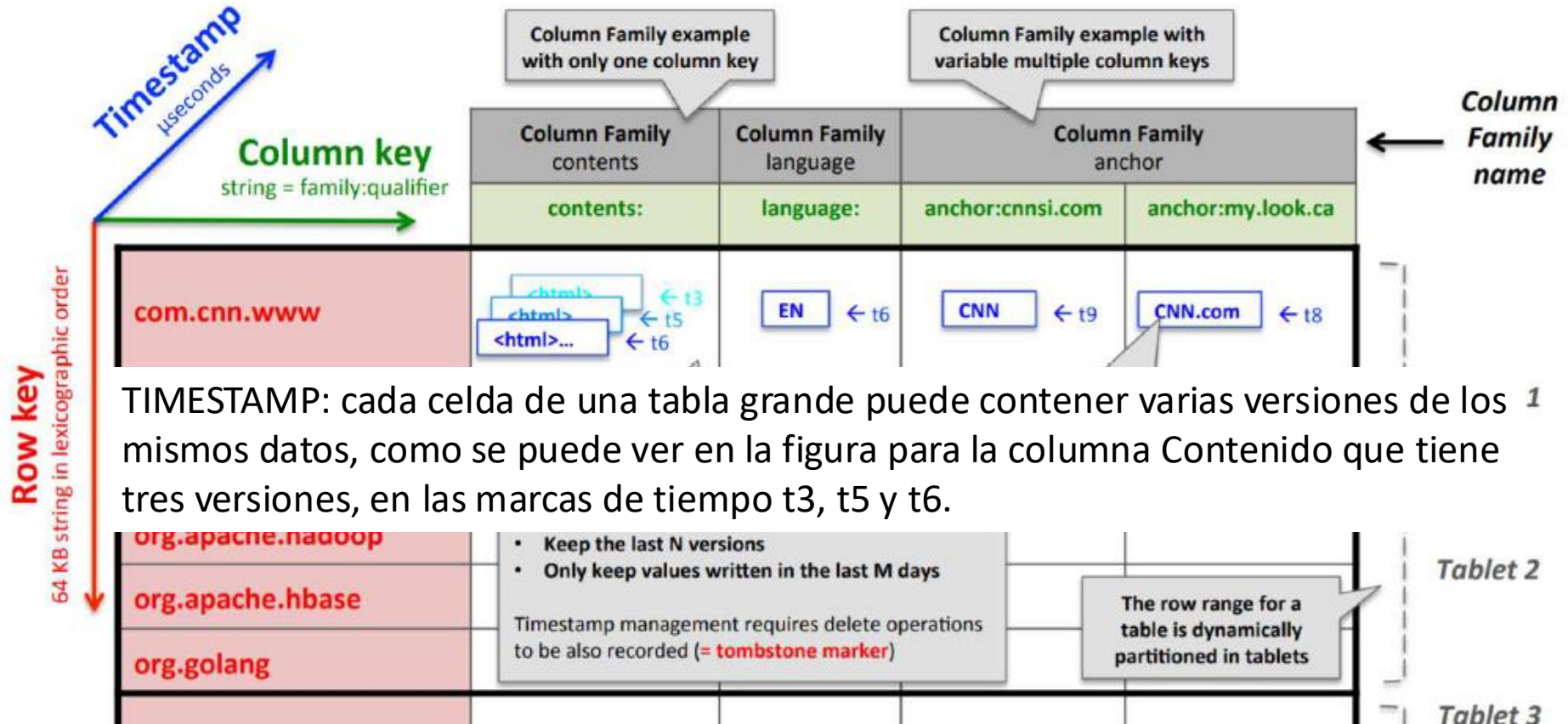
t 2

t 3

Bigtable: Modelo de datos



Bigtable: Modelo de datos



Bigtable: resumen

(row, column, time) → value

- row: a row id string
 - e.g., “Afghanistan”
- column: a column name string
 - e.g., “pop-value”
- time: an integer (64-bit) version time-stamp
 - e.g., 18545664
- value: the element of the cell
 - e.g., “31120978”

Bigtable: resumen

¡Valor de clave principal solamente!

(row, column, time) $\rightarrow$ value

(Afghanistan, pop-value, t_4) $\rightarrow$ 31108077

Primary Key	capital		continent		pop-value		pop-year	
Afghanistan	t ₁	Kabul	t ₁	Asia	t ₁	31143292	t ₁	2009
					t ₂	31120978		
					t ₄	31108077	t ₄	2011
Albania	t ₁	Tirana	t ₁	Europe	t ₁	2912380	t ₁	2010
					t ₃	3011405	t ₃	2013
...	...		...		...		...	

Bigtable: Claves ordenadas



Primary Key	capital		pop-value		pop-year	
Asia:Afghanistan	t ₁	Kabul	t ₁	31143292	t ₁	2009
			t ₂	31120978		
			t ₄	31108077	t ₄	2011
Asia:Azerbaijan	...	...	...	...	...	...
...	...	...	...	...	...	...
Europe:Albania	t ₁	Tirana	t ₁	2912380	t ₁	2010
			t ₃	3011405	t ₃	2013
Europe:Andorra	...	...	...	...	...	...
...	...	...	...	...	...	...

¿Beneficios de las llaves ordenadas frente a las llaves hash? (?)

Consultas de rango y...

Bigtable: Tablas

	Primary Key	capital		pop-value		pop-year	
ASIA	Asia:Afghanistan	t ₁	Kabul	t ₁	31143292	t ₁	2009
				t ₂	31120978		
				t ₄	31108077	t ₄	2011
	Asia:Azerbaijan	...	...	...	...	...	...
	...	...	...	...	...	...	...
EUROPE	Europe:Albania	t ₁	Tirana	t ₁	2912380	t ₁	2010
				t ₃	3011405	t ₃	2013
	Europe:Andorra	...	...	...	...	...	...
	...	...	...	...	...	...	...

¿Beneficios de las llaves ordenadas frente a las llaves hash?



Consultas de rango y...

... localidad de procesamiento

Un ejemplo del mundo real de localidad/clasificación



Primary Key	language		title		links	
com.imdb	t ₁	en	t ₁	IMDb Home	t ₁	...
			t ₂	IMDB - Movies		
			t ₄	IMDb	t ₄	...
com.imdb/title/tt2724064/	t ₁	en	t ₂	Sharknado	t ₂	...
com.imdb/title/tt3062074/	t ₁	en	t ₂	Sharknado II	t ₂	...
...	...	...	...	...	...	...
org.wikipedia	t ₁	multi	t ₁	Wikipedia	t ₁	...
			t ₃	Wikipedia Home	t ₃	...
org.wikipedia.ace	t ₁	ace	t ₁	Wikipèdia bahsa Acèh	...	...
...	...	...	...	...	...	...



Bigtable: Distribution

Asia:Afghanistan	t ₁	Kabul	t ₁	31143292	t ₁	2009
			t ₂	31120978		
			t ₄	31108077	t ₄	2011
Asia:Azerbaijan	...	...	...	...	...	...
...	...	...	...	...	...	...

Europe:Albania	t ₁	Tirana	t ₁	2912380	t ₁	2010
			t ₃	3011405	t ₃	2013
Europe:Andorra	...	...	...	...	...	...
...	...	...	...	...	...	...

Dividir por tabla



Partición de rango horizontal

Bigtable: Column Families

Primary Key	pol:capital		demo:pop-value		demo:pop-year	
Asia:Afghanistan	t ₁	Kabul	t ₁	31143292	t ₁	2009
			t ₂	31120978		
			t ₄	31108077	t ₄	2011
Asia:Azerbaijan	...	...	...	...	...	...
...	...	...	...	...	...	...
Europe:Albania	t ₁	Tirana	t ₁	2912380	t ₁	2010
			t ₃	3011405	t ₃	2013
Europe:Andorra	...	...	...	...	...	...
...	...	...	...	...	...	...

- Agrupar columnas lógicamente similares juntas
 - Accedidos juntos de manera eficiente
 - Control de acceso y almacenamiento: nivel de familia de columnas
 - Si es del mismo tipo, puede comprimirse

Bigtable: Versionado

- Similar a Apache Cassandra
 - Cell-level
 - Marcas de tiempo de enteros de 64 bits
 - Las inserciones empujan hacia abajo la versión actual
 - Eliminaciones perezosas/recolección de basura periódica
 - Dos opciones:
 - Mantener las últimas n versiones
 - Mantener las versiones más nuevas que el tiempo t

Bigtable: SSTable Map Implementation

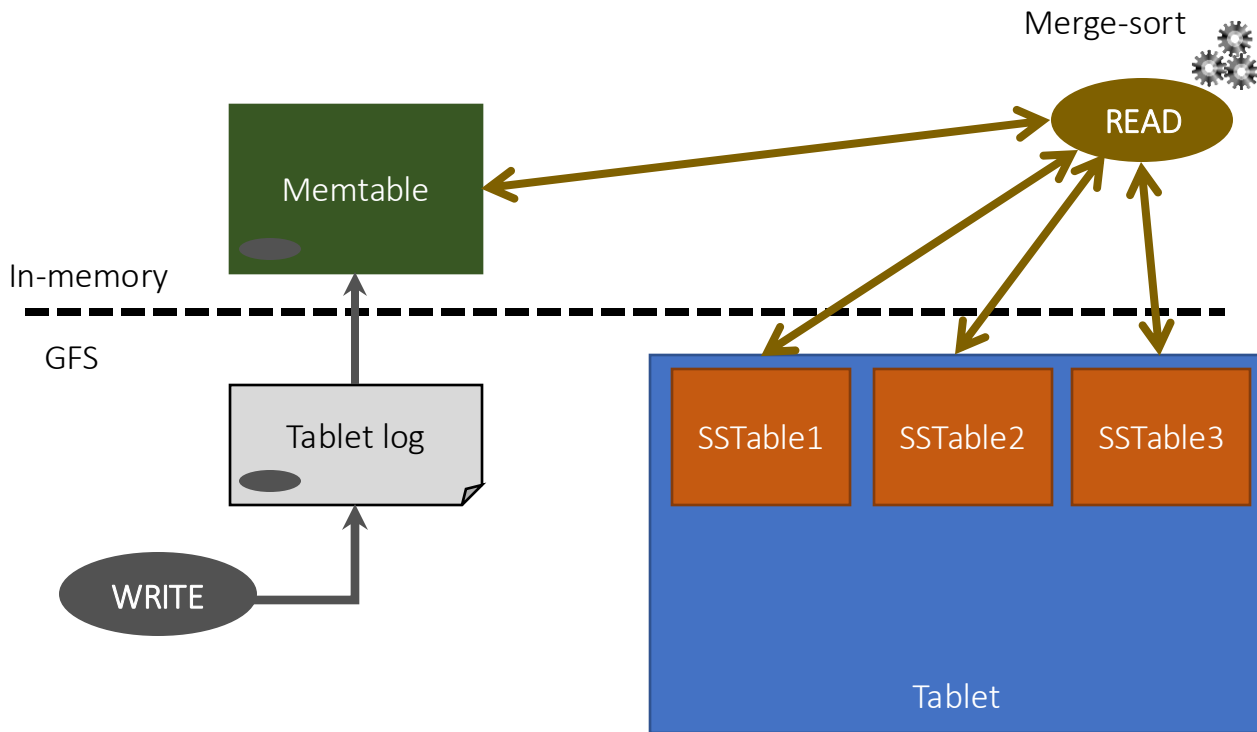
- Bloques de 64k (predeterminado) con índice en el pie de página (GFS)
- Índice cargado en la memoria, permite búsquedas
- Se puede dividir o fusionar, según sea necesario

Escrituras?

	Primary Key	pol:capital		demo:pop-value		demo:pop-year	
0	Asia:Afghanistan	t ₁	Kabul	t ₁	31143292	t ₁	2009
				t ₂	31120978		
				t ₄	31108077	t ₄	2011
	Asia:Azerbaijan	...	...	...	...	...	...
...	...	...	...	...	...	...	
65536	Asia:Japan	...	...	...	...	...	...
	Asia:Jordan	...	...	...	...	...	...
	...	...	...	...	...	...	...
	Block 0 / Offset 0 / Asia:Afghanistan						
Index:	Block 1 / Offset 65536 / Asia: Japan						

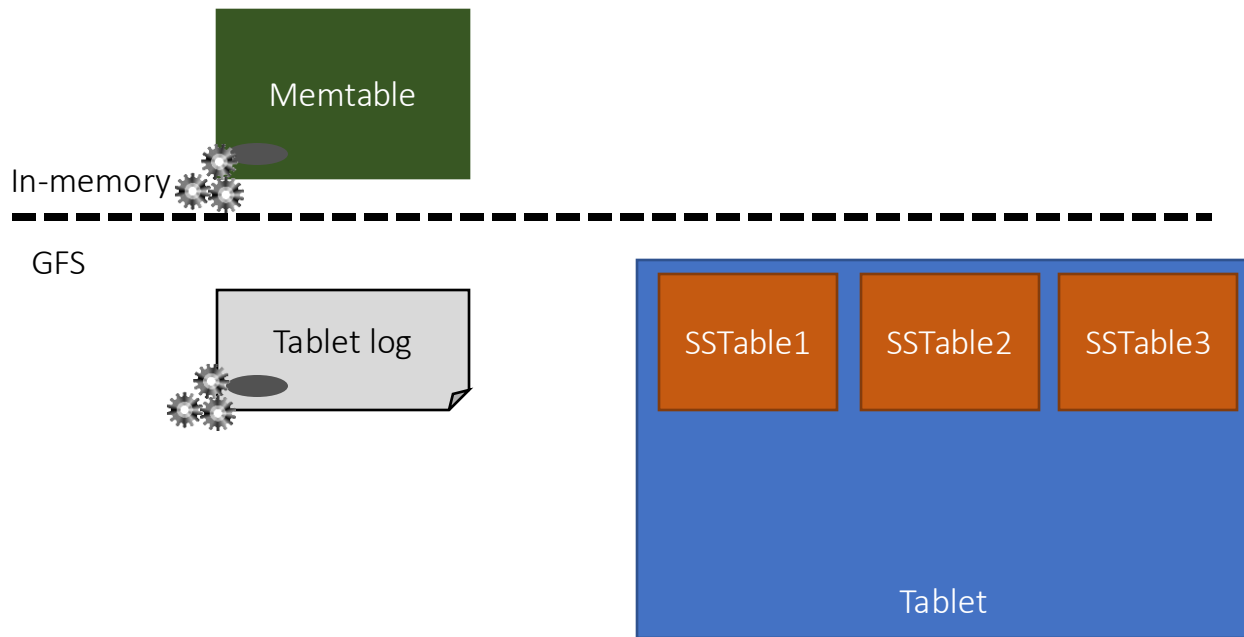
Bigtable: Escrituras en búfer/por lotes

¿Cuál es el peligro?



Bigtable: Redo Log (Rehacer registro)

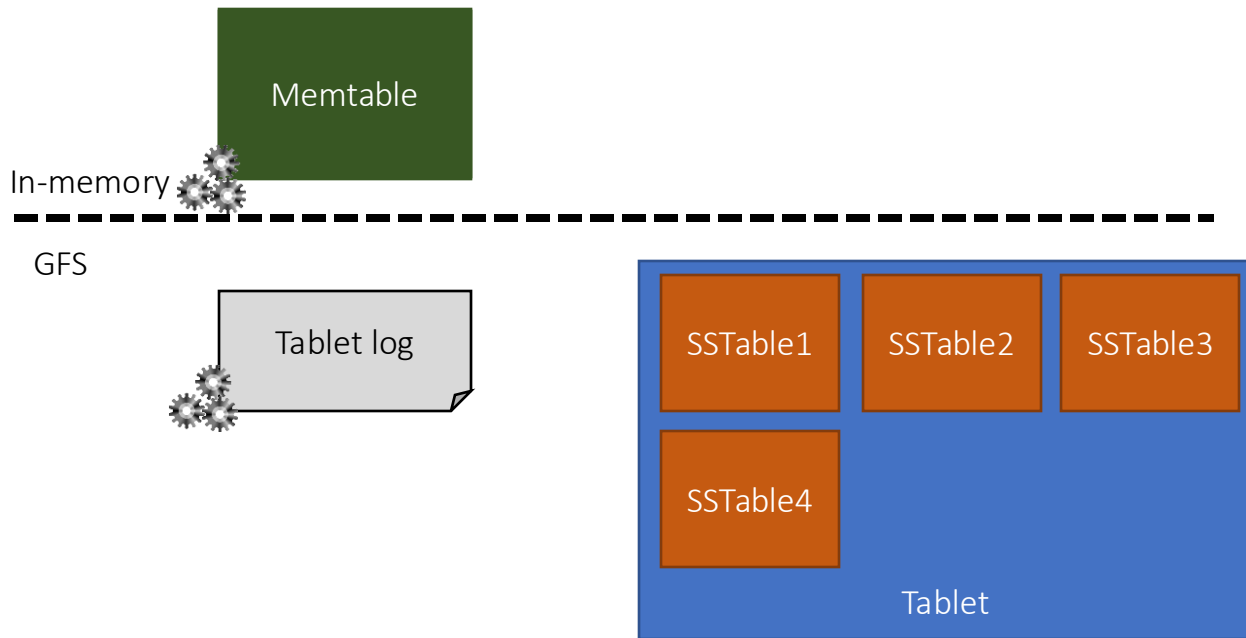
- Si la máquina falla, Memtable se rehace desde el registro



Bigtable: Compactación Menor

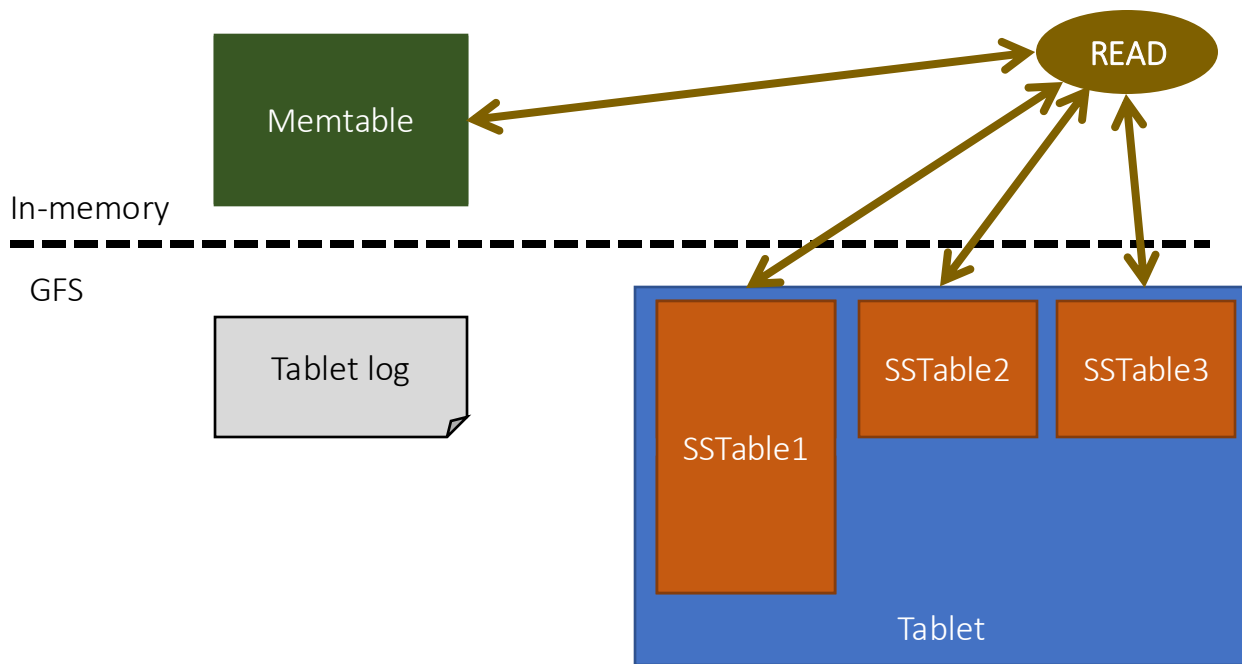
- Cuando esté llena, escriba Memtable

¿Problema con el rendimiento? (?)



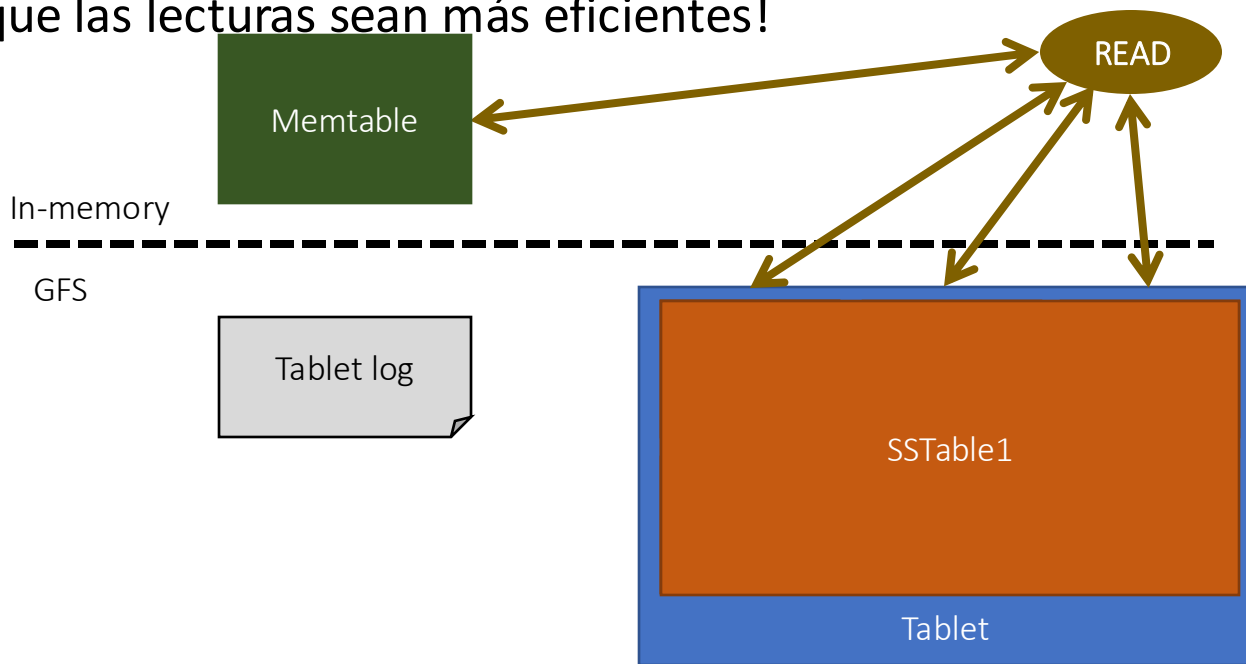
Bigtable: Fusionar compactación

- Combinar algunas de las SSTables (y la Memtable)



Bigtable: Compactación Mayor

- Combinar todas las SSTables (y la Memtable)
- ¡Hace que las lecturas sean más eficientes!

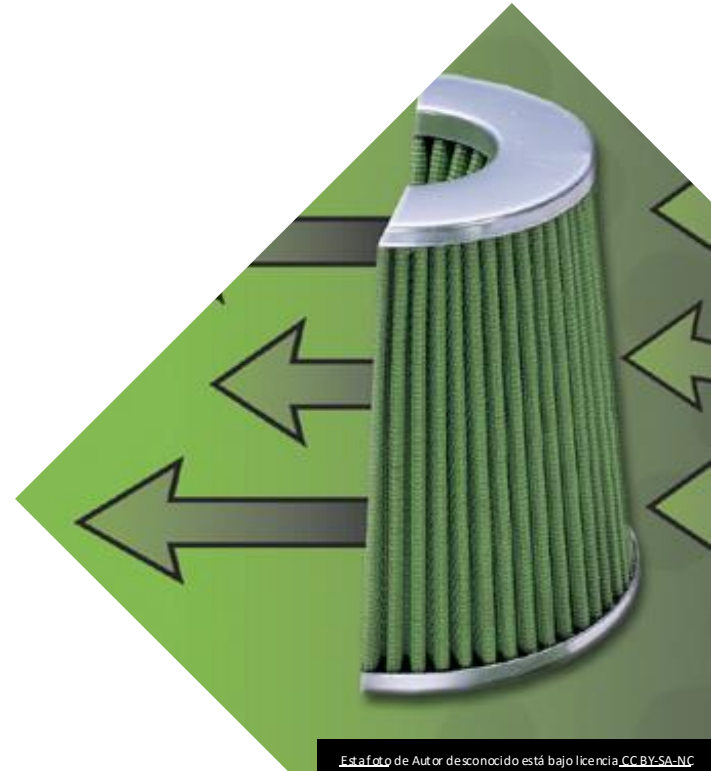


Bigtable: Un montón de otras cosas

- **Hierarchy and locks**: cómo encontrar y bloquear tablas
- **Locality groups**: Agrupe varias familias de columnas juntas; asignado un SSTable separado
- **Select storage**: las SSTables pueden ser persistentes o en memoria
- **Compression**: Aplicado en bloques SSTable; se puede elegir una compresión personalizada
- **Caches**: SSTable-level y block-level
- **Bloom filters**: Encuentra negativos a bajo costo...

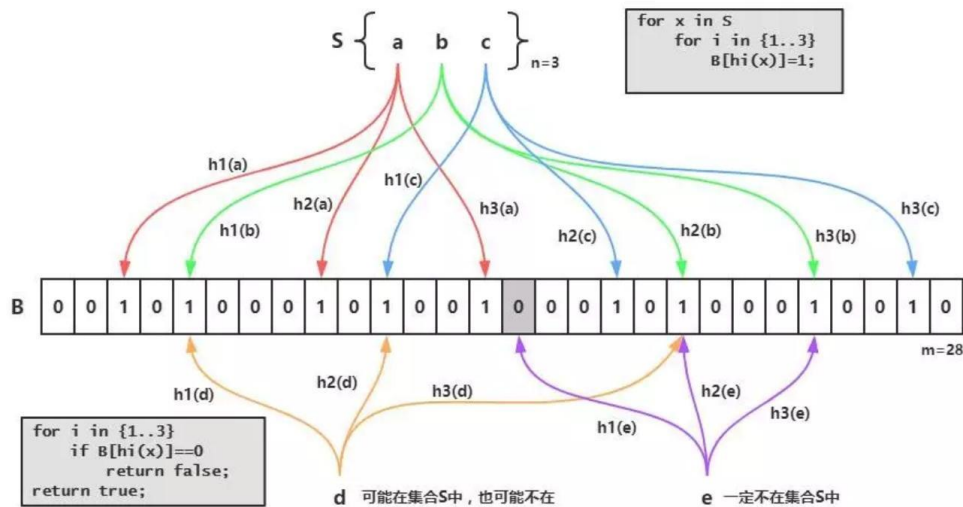
Filtro Bloom

- El filtro bloom es una estructura de datos probabilística basada en funciones de hashing, que provee una representación compacta de un conjunto de elementos (llaves).
- Esta estructura permite la inserción y búsqueda de elementos.
- Una respuesta ante la búsqueda de algún elemento tiene dos posibles resultados: falsos positivos o negativos.

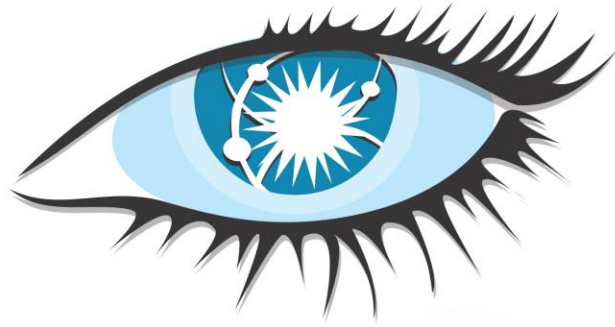


Filtro Bloom

- Primero hay que entender que este filtro es de un tamaño N de bits, y presenta una cantidad K de funciones hash.
- Esto influye en el desempeño y en la eficacia del filtro.



Apache Cassandra



Modelo de Datos

- Diseñado para datos distribuidos de modo escalable → sacrifica ACID por ventajas en rendimiento, disponibilidad y gestión operacional.
- Los modelos que se crean son desnormalizados:
 - Se suele crear una column family por cada consulta (query) a realizar
 - Varias filas en un column family suelen dar respuesta a una consulta

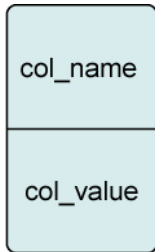
Modelo de Datos

- Los conceptos básicos son:
- **Clúster**: son las máquinas que componen una instancia de Cassandra
 - Pueden contener varios Keyspaces
- **Keyspace**: espacio de nombres para un conjunto de ColumnFamily, asociado a una aplicación
 - Suele vincularse con una BBDD en el modelo relacional
- **ColumnFamily**: contienen varias columnas
 - Suelen vincularse con una tabla en el modelo relacional
- **SuperColumn**: columnas que ellas mismas tienen sub-columnas
- **Column**: compuestas de un nombre, valor y timestamp

Modelo de Datos

- Column

- Una **columna** es un par nombre-valor que también contiene un timestamp.
- Los nombres y columnas son arrays de bytes
- El timestamp registra la última vez que una columna es accedida
- Unidad atómica
 - `name:value:timestamp`
 - `Email:dipina@deusto.es:123456789`
- Ejemplo en JSON:
 - `{`
 - `"name": "Email",`
 - `"value": "dipina@deusto.es",`
 - `"timestamp": 123456789`



Modelo de Datos

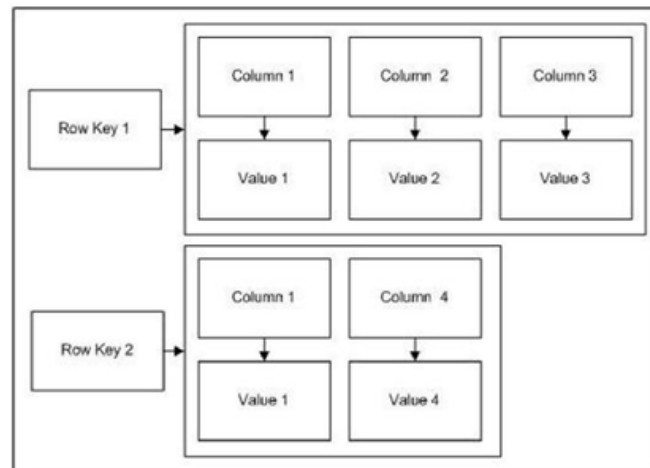
- Column Family

- Es un contenedor de columnas
 - Análogo al concepto de tabla en RDBMS
- Contiene una lista ordenada de columnas
 - Cuando se crea se indica cómo se ordenarán las columnas de cada fila
 - ASCII, UTF-8, Long, UUID

Modelo de Datos

- Column Family

- Cada familia de columnas se guarda en un fichero, y el fichero está ordenado por clave de fila
- Una familia de columnas tiene un ...
 - conjunto de filas con un conjunto de ...
 - columnas similar pero no idéntico
- Pueden ser de tipo SUPER o STANDARD



Modelo de Datos

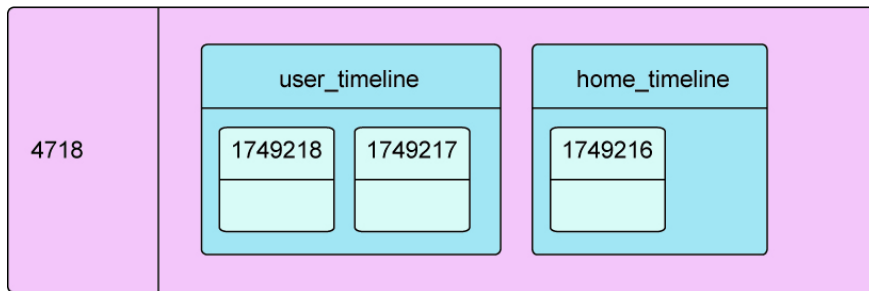
- Componentes

- Espacio de Claves (Keyspace): Contenedor más externo para los datos en Cassandra. Se podría considerar como un análogo a una base de datos en un modelo relacional. Hay tres atributos que se pueden definir por espacio de claves:
 - Factor de replicación.
 - Posicionamiento de las réplicas.
 - Familia de Columnas.

Modelo de Datos

- KeySpaces

- Un espacio de claves o KeySpace es un esquema de alto nivel que contiene familias de columnas.
 - Supercolumn Family “Estado de Usuario”



Modelo de Datos

- KeySpaces

- Column Family “Entradas en Twitter: tweets”:

1749217	text	user_id
	tweet tweet tweet	4718

1749218	text	user_id	in_reply
	@nakhli blah blah	4718	1749216

Modelo de Datos

- Lenguaje de Consultas de Cassandra (CQL)

Definición de datos:

- Create keyspace

Sintaxis:

```
create_keyspace_statement ::= CREATE KEYSPACE [ IF NOT EXISTS ] keyspace_name WITH options
```

Fuente: cassandra.apache.org

Ejemplo:

```
CREATE KEYSPACE Excelsior  
    WITH replication = {'class': 'SimpleStrategy', 'replication_factor' : 3};
```

Fuente: cassandra.apache.org

Modelo de Datos

- Lenguaje de Consultas de Cassandra (CQL)

Definición de datos:

- Alter Keyspace

Sintaxis:

```
alter_keyspace_statement ::= ALTER KEYSPACE keyspace_name WITH options
```

Fuente: cassandra.apache.org

Ejemplo:

```
ALTER KEYSPACE Excelsior  
  WITH replication = {'class': 'SimpleStrategy', 'replication_factor' : 4};
```

Fuente: cassandra.apache.org

Modelo de Datos

- Lenguaje de Consultas de Cassandra (CQL)

Definición de datos:

- Drop Keyspace

Sintaxis:

```
drop_keyspace_statement ::= DROP KEYSPACE [ IF EXISTS ] keyspace_name
```

Fuente: cassandra.apache.org

Ejemplo:

```
DROP KEYSPACE Excelsior;
```

Fuente: cassandra.apache.org

Modelo de Datos

- Lenguaje de Consultas de Cassandra (CQL)

Definición de datos:

- Create table

Sintaxis:

```
create_table_statement ::= CREATE TABLE [ IF NOT EXISTS ] table_name  
                        '('  
                        column_definition  
                        ( ',' column_definition )*  
                        [ ',' PRIMARY KEY '(' primary_key ')' ]  
                        ')' [ WITH table_options ]
```

Ejemplo:

Fuente: cassandra.apache.org

```
CREATE TABLE monkeySpecies (  
    species text PRIMARY KEY,  
    common_name text,  
    population varint,  
    average_size int  
) WITH comment='Important biological records'  
    AND read_repair_chance = 1.0;
```

Fuente: cassandra.apache.org

Modelo de Datos

- Lenguaje de Consultas de Cassandra (CQL)

Definición de datos:

- Alter table

Sintaxis:

```
alter_table_statement ::= ALTER TABLE table_name alter_table_instruction
alter_table_instruction ::= ADD column_name cql_type ( ',' column_name cql_type )*
                        | DROP column_name ( column_name )*
                        | WITH options
```

Fuente: cassandra.apache.org

Ejemplo:

```
ALTER TABLE addamsFamily ADD gravesite varchar;
```

Fuente: cassandra.apache.org

Modelo de Datos

- Lenguaje de Consultas de Cassandra (CQL)

Definición de datos:

- Drop table

Sintaxis:

```
drop_table_statement ::= DROP TABLE [ IF EXISTS ] table_name
```

Fuente: cassandra.apache.org

Modelo de Datos

- Lenguaje de Consultas de Cassandra (CQL)

Definición de datos

- Truncate

Sintaxis:

```
truncate_statement ::= TRUNCATE [ TABLE ] table_name
```

Fuente: cassandra.apache.org

Modelo de Datos

- Lenguaje de Consultas de Cassandra (CQL)

Tipos de datos → En Cassandra existen tipos de datos primitivos y también pueden ser definidos por el usuario.

type	constants supported	description
<code>ascii</code>	<code>string</code>	ASCII character string
<code>bigint</code>	<code>integer</code>	64-bit signed long
<code>blob</code>	<code>blob</code>	Arbitrary bytes (no validation)
<code>boolean</code>	<code>boolean</code>	Either <code>true</code> or <code>false</code>
<code>counter</code>	<code>integer</code>	Counter column (64-bit signed value). See Counters for details
<code>date</code>	<code>integer</code> , <code>string</code>	A date (with no corresponding time value). See Working with dates below for details
<code>decimal</code>	<code>integer</code> , <code>float</code>	Variable-precision decimal
<code>double</code>	<code>integer</code> , <code>float</code>	64-bit IEEE-754 floating point
<code>duration</code>	<code>duration</code> , <code></code>	A duration with nanosecond precision. See Working with durations below for details
<code>float</code>	<code>integer</code> , <code>float</code>	32-bit IEEE-754 floating point
<code>inet</code>	<code>string</code>	An IP address, either IPv4 (4 bytes long) or IPv6 (16 bytes long). Note that there is no <code>inet</code> constant, IP address should be input as strings
<code>int</code>	<code>integer</code>	32-bit signed int
<code>smallint</code>	<code>integer</code>	16-bit signed int
<code>text</code>	<code>string</code>	UTF8 encoded string
<code>time</code>	<code>integer</code> , <code>string</code>	A time (with no corresponding date value) with nanosecond precision. See Working with times below for details
<code>timestamp</code>	<code>integer</code> , <code>string</code>	A timestamp (date and time) with millisecond precision. See Working with timestamps below for details
<code>timeuuid</code>	<code>uuid</code>	Version 1 UUID , generally used as a "conflict-free" timestamp. Also see Timeuuid functions
<code>tinyint</code>	<code>integer</code>	8-bit signed int
<code>uuid</code>	<code>uuid</code>	A UUID (of any version)
<code>varchar</code>	<code>string</code>	UTF8 encoded string
<code>varint</code>	<code>integer</code>	Arbitrary-precision integer

Figura 2.9: Tipos de datos nativos en Cassandra. (Foundation, 2016)

Modelo de Datos

- Lenguaje de Consultas de Cassandra (CQL)

Manipulación de Datos

- Sentencia Select

Sintaxis:

```
select_statement ::= SELECT [ JSON | DISTINCT ] ( select_clause | '*' )  
                  FROM table_name  
                  [ WHERE where_clause ]  
                  [ GROUP BY group_by_clause ]  
                  [ ORDER BY ordering_clause ]  
                  [ PER PARTITION LIMIT (integer | bind_marker) ]  
                  [ LIMIT (integer | bind_marker) ]  
                  [ ALLOW FILTERING ]
```

Fuente: cassandra.apache.org

Ejemplo:

```
SELECT name, occupation FROM users WHERE userid IN (199, 200, 207);  
SELECT JSON name, occupation FROM users WHERE userid = 199;  
SELECT name AS user_name, occupation AS user_occupation FROM users;  
  
SELECT time, value  
FROM events  
WHERE event_type = 'myEvent'  
   AND time > '2011-02-03'  
   AND time <= '2012-01-01'  
  
SELECT COUNT (*) AS user_count FROM users;
```

Fuente: cassandra.apache.org

Modelo de Datos

- Lenguaje de Consultas de Cassandra (CQL)

Manipulación de Datos

- Sentencia Insert

Sintaxis:

```
insert_statement ::= INSERT INTO table_name ( names_values | json_clause )  
                  [ IF NOT EXISTS ]  
                  [ USING update_parameter ( AND update_parameter )* ]  
names_values      ::= names VALUES tuple_literal  
json_clause       ::= JSON string [ DEFAULT ( NULL | UNSET ) ]  
names             ::= '(' column_name ( ',' column_name )* ')'
```

Fuente: cassandra.apache.org

Ejemplo:

```
SELECT name, occupation FROM users WHERE userid IN (199, 200, 207);  
SELECT JSON name, occupation FROM users WHERE userid = 199;  
SELECT name AS user_name, occupation AS user_occupation FROM users;  
  
SELECT time, value  
FROM events  
WHERE event_type = 'myEvent'  
   AND time > '2011-02-03'  
   AND time <= '2012-01-01'  
  
SELECT COUNT (*) AS user_count FROM users;
```

Fuente: cassandra.apache.org

Modelo de Datos

- Lenguaje de Consultas de Cassandra (CQL)

Manipulación de Datos

- Sentencia Update

Sintaxis:

```
update_statement ::= UPDATE table_name
                  [ USING update_parameter ( AND update_parameter )* ]
                  SET assignment ( ',' assignment )*
                  WHERE where_clause
                  [ IF ( EXISTS | condition ( AND condition )* ) ]
update_parameter ::= ( TIMESTAMP | TTL ) ( integer | bind_marker )
assignment       ::= simple_selection '=' term
                  | column_name '=' column_name ( '+' | '-' ) term
                  | column_name '=' list_literal '+' column_name
simple_selection  ::= column_name
                  | column_name '[' term ']'
                  | column_name '.' field_name
condition        ::= simple_selection operator term
```

Fuente: cassandra.apache.org

Ejemplo:

```
UPDATE NerdMovies USING TTL 400
SET director = 'Joss Whedon',
    main_actor = 'Nathan Fillion',
    year = 2005
WHERE movie = 'Serenity';

UPDATE UserActions
SET total = total + 2
WHERE user = B70DE1D0-9908-4AE3-BE34-5573E5B09F14
AND action = 'click';
```

Fuente: cassandra.apache.org

Modelo de Datos

- Lenguaje de Consultas de Cassandra (CQL)

Manipulación de Datos

- Sentencia Delete

Sintaxis:

```
delete_statement ::= DELETE [ simple_selection ( ',' simple_selection ) ]  
                  FROM table_name  
                  [ USING update_parameter ( AND update_parameter )* ]  
                  WHERE where_clause  
                  [ IF ( EXISTS | condition ( AND condition )* ) ]
```

Fuente: cassandra.apache.org

Ejemplo:

```
DELETE FROM NerdMovies USING TIMESTAMP 1240003134  
WHERE movie = 'Serenity';  
  
DELETE phone FROM Users  
WHERE userid IN (C73DE1D3-AF08-40F3-B124-3FF3E5109F22, B70DE1D0-9908-4AE3-BE34-5573E5B09F14);
```

Fuente: cassandra.apache.org

Modelo de Datos

- Lenguaje de Consultas de Cassandra (CQL)

Manipulación de Datos

- Sentencia Batch

Sintaxis:

```
batch_statement      ::= BEGIN [ UNLOGGED | COUNTER ] BATCH  
                        [ USING update_parameter ( AND update_parameter )* ]  
                        modification_statement ( ';' modification_statement )*  
                        APPLY BATCH  
modification_statement ::= insert_statement | update_statement | delete_statement
```

Fuente: cassandra.apache.org

Ejemplo:

```
BEGIN BATCH  
  INSERT INTO users (userid, password, name) VALUES ('user2', 'ch@ngem3b', 'second user');  
  UPDATE users SET password = 'ps22dhds' WHERE userid = 'user3';  
  INSERT INTO users (userid, password) VALUES ('user4', 'ch@ngem3c');  
  DELETE name FROM users WHERE userid = 'user1';  
APPLY BATCH;
```

Fuente: cassandra.apache.org

Arquitectura

- Cassandra es un sistema distribuido; usa sistemas de clúster.
- Utiliza el modelo P2P; todos los nodos se comportan como iguales.
- Ningún nodo tiene control sobre los demás, haciendo que Cassandra también sea un sistema descentralizado.
- Los nodos utilizan el protocolo Gossip para comunicarse con sus vecinos.

Arquitectura

- Conceptos

- CommitLog: Registro de todos los cambios en un nodo.
- MemTables: Estructuras en memoria en dónde los búffers escriben.
- SSTables: Archivos de datos que se utilizan para la persistencia de datos en disco.

Arquitectura

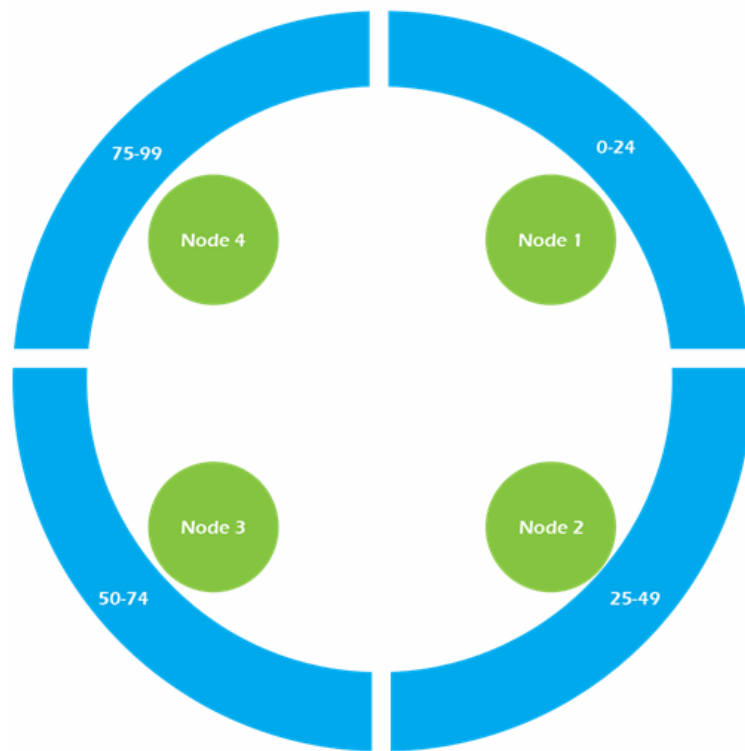
- Particionamiento

- Cassandra organiza los datos en particiones.
- Cada partición se guarda en un nodo.
- Un nodo puede poseer una o más particiones.

Dos tipos: Ordenado y Aleatorio.

- Ordenado: Los datos se particionan de manera ordenada.
- Aleatorio: A los datos se les aplica una función hash para saber a cuál partición se asignan.

Ejemplo de la distribución de las particiones en un clúster de 4 nodos.



Fuente:
<https://shermadigital.com/blog/designing-a-cassandra-data-model/>

Arquitectura Replicación

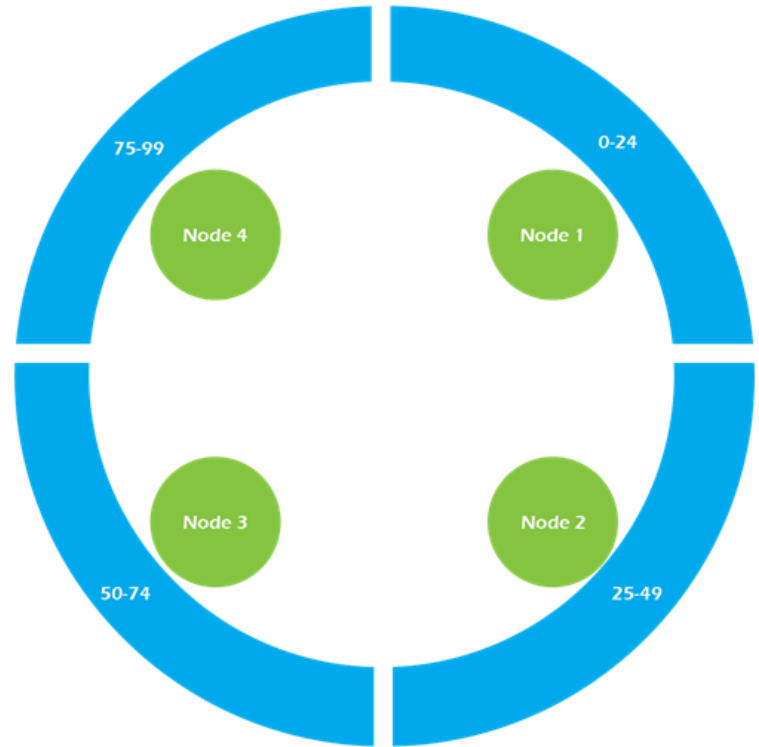
Consiste en:

- Asegurar confiabilidad y tolerancia a fallos, por medio de réplicas en múltiples nodos.
- Factor de replicación.
- Estrategia de replicación.

Arquitectura Replicación

Estrategia de replicación simple:

- Solo un centro de datos.
- Ambiente de desarrollo.
- Utiliza la llave de partición.



Fuente:

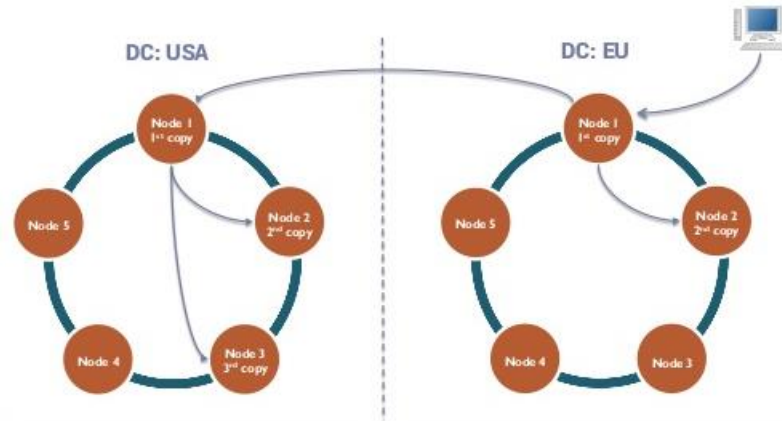
<https://shermadigital.com/blog/designing-a-cassandra-data-model/>

Arquitectura Replicación

Estrategia de replicación de topología de red:

- Ambiente de producción.
- Factor de replicación.
- Distintos centros de datos.
- Consistencia configurable.

```
CREATE KEYSPACE johnny WITH REPLICATION =  
{'class':'NetworkTopologyStrategy', 'USA':3, 'EU': 2};
```



Fuente: <https://docs.datastax.com>

Acceso Cliente

- Acceso por medio de un API

La API de Apache Cassandra es CQL. Para hacer uso de la API, hay que conectarse con el clúster, lo cual puede hacerse de dos formas:

- La primera es haciendo uso de CQLSH.
- La segunda opción es por medio de drivers de cliente.